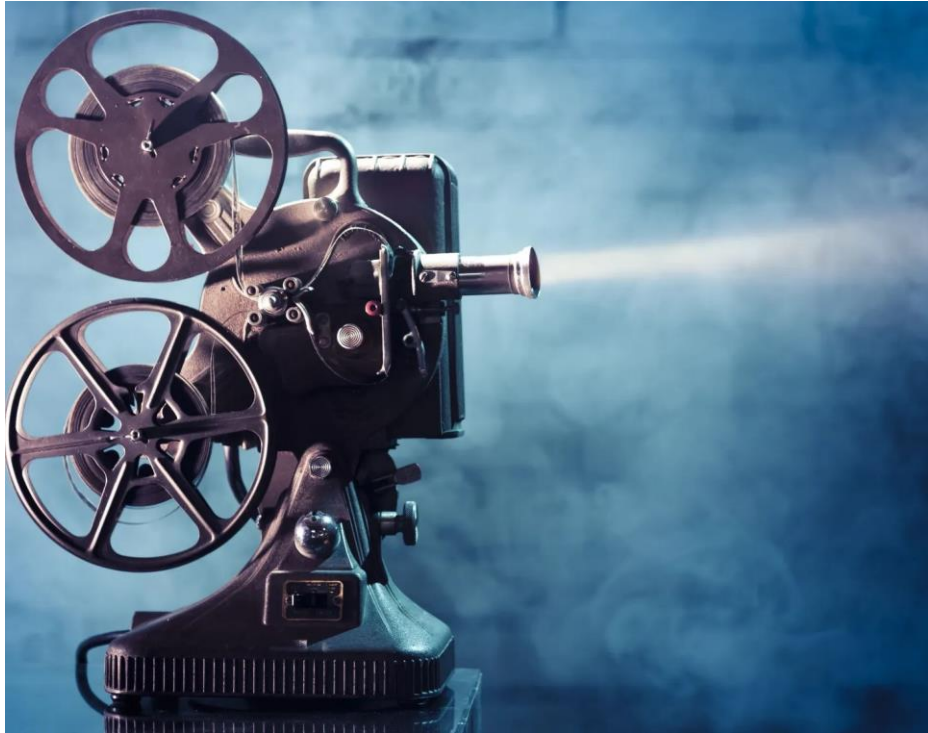




# CAB230 ASSIGNMENT 2

## CLIENT SIDE

Movies API – Client Side Application

Rory Blair  
N10773398

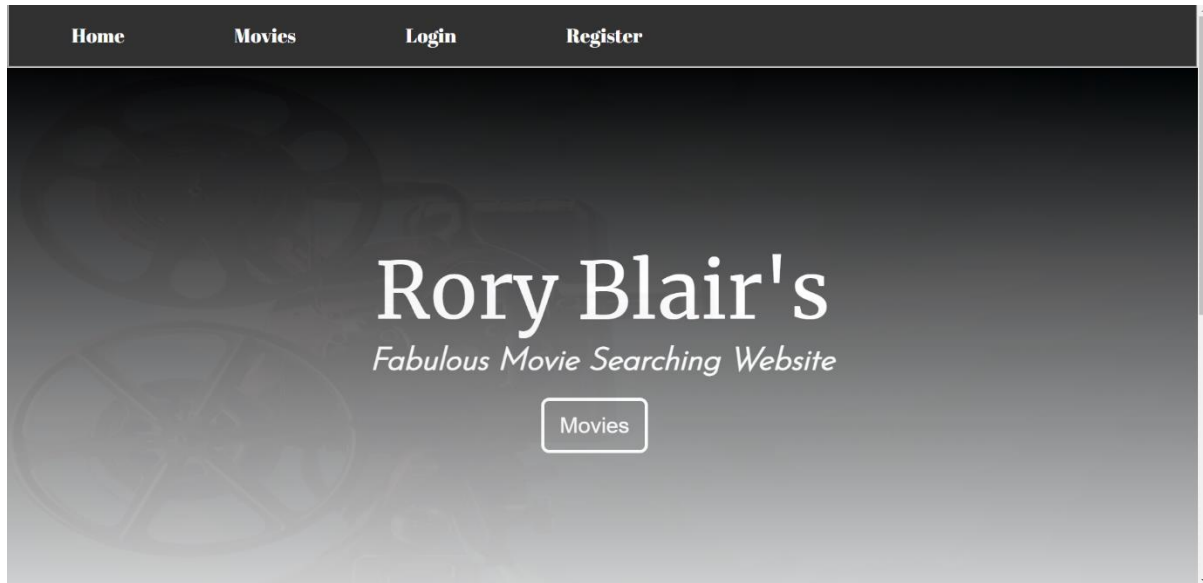
# Table of Contents

|                                               |                                     |
|-----------------------------------------------|-------------------------------------|
| <b>Introduction.....</b>                      | <b>2</b>                            |
| Purpose and Description.....                  | 2                                   |
| Completeness and limitations.....             | 2                                   |
| <b>Use of Endpoints.....</b>                  | <b>3</b>                            |
| /movies/search.....                           | 3                                   |
| /movies/data/{imdbID}.....                    | 4                                   |
| /people/{id}.....                             | 5                                   |
| /user/register .....                          | 6                                   |
| /user/login.....                              | 7                                   |
| /user/refresh .....                           | 7                                   |
| /user/logout.....                             | 8                                   |
| <b>Modules Used.....</b>                      | <b>9</b>                            |
| Module 1: Ag-Grid.....                        | 9                                   |
| Module 1.1: Ag-Grid-Community.....            | 9                                   |
| Module 1.2: Ag-Grid-React .....               | 9                                   |
| Module 2: Bootstrap .....                     | 9                                   |
| Module 3: React.....                          | 9                                   |
| Module 4: React-Bootstrap .....               | 9                                   |
| Module 5: React-Dom .....                     | 10                                  |
| Module 6: React-Router-Dom .....              | 10                                  |
| Module 7: React-Scripts .....                 | 10                                  |
| Module 8: Reactstrap.....                     | 10                                  |
| <b>Application Design and Usability .....</b> | <b>11</b>                           |
| Navigation and Layout.....                    | 11                                  |
| Usability and Quality of Design .....         | 13                                  |
| Accessibility.....                            | 14                                  |
| <b>Technical Description .....</b>            | <b>15</b>                           |
| Architecture .....                            | 15                                  |
| Test Plan .....                               | 17                                  |
| Problems .....                                | 18                                  |
| <b>Extensions .....</b>                       | <b>Error! Bookmark not defined.</b> |
| <b>User Guide.....</b>                        | <b>19</b>                           |
| <b>References.....</b>                        | <b>20</b>                           |
| <b>Appendices.....</b>                        | <b>22</b>                           |

# Introduction

## Purpose and Description

This is an essentially a fully functioning web application that allows user to browse the movies database, select to view more details on a movie and to see a particular person's profile. You are able to browse movies by searching either the title or year or both. You are able to view more details on a particular movie by clicking on the movie on which you wish to view. You are able to view a person's profile in a similar way.



*Figure 1: Home Page*

## Completeness and limitations

The web application would be considered a grade level of 5 as the data endpoint all work as intended. The '/movies/search' gives all movies with a set of particular queries. The '/movie/data/id' endpoint give all the information about a particular movie. The '/people/id' gives out all data on a particular person if the user is authenticated. The authentication endpoint also work, to an extent. There is no excessive querying of the server, and the web application isn't hard to use. The Ag-grid is used to create tables quickly and efficiently, whilst having infinite scrolling. Navigation between pages is done via the React Router Library.

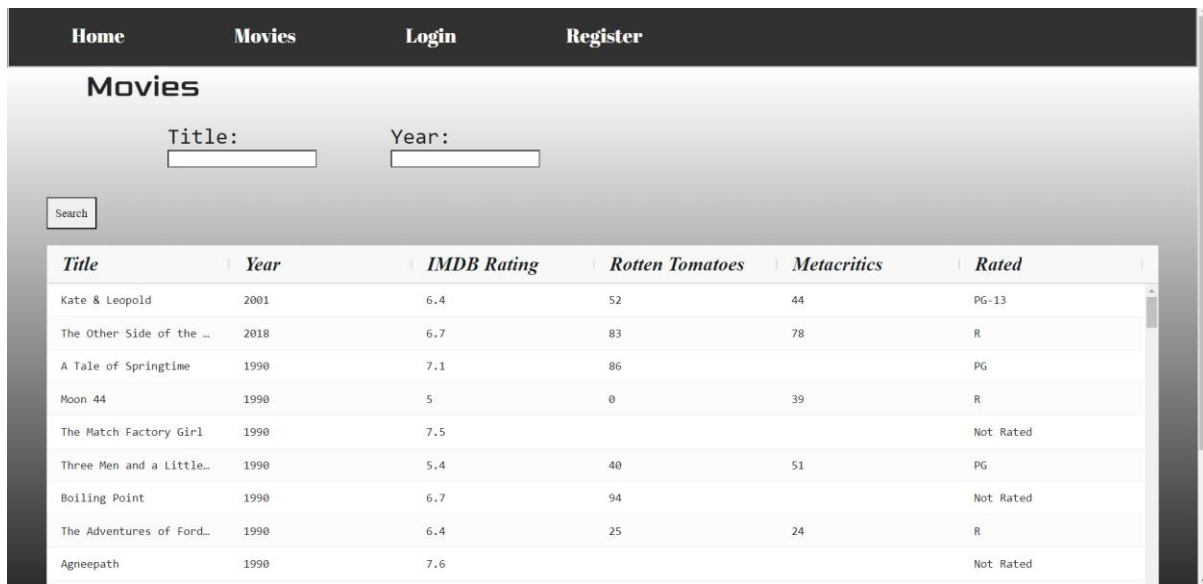
# Use of Endpoints

## /movies/search

This endpoint is used to retrieve the data from the server via a fetch command that uses a URL to retrieve the data. It retrieves the data and stores it as a 'json', where the necessary data can be pulled from it. The URL used takes the form:

<http://sefdb02.gut.edu.au:3000/movies/search?page=PAGE&title=TITLE&year=YEAR>

This endpoint always has the variable 'PAGE' as 1 and the 'TITLE' and 'YEAR' variables are the search parameters being used. If one is null it will complete the search as if it wasn't queried.



The screenshot shows a web application with a dark navigation bar containing links for Home, Movies, Login, and Register. Below the navigation bar, the page is titled "Movies". There is a search form with input fields for "Title:" and "Year:", and a "Search" button. Below the search form is a table with the following data:

| Title                     | Year | IMDB Rating | Rotten Tomatoes | Metacritics | Rated     |
|---------------------------|------|-------------|-----------------|-------------|-----------|
| Kate & Leopold            | 2001 | 6.4         | 52              | 44          | PG-13     |
| The Other Side of the ... | 2018 | 6.7         | 83              | 78          | R         |
| A Tale of Springtime      | 1990 | 7.1         | 86              |             | PG        |
| Moon 44                   | 1990 | 5           | 0               | 39          | R         |
| The Match Factory Girl    | 1990 | 7.5         |                 |             | Not Rated |
| Three Men and a Little... | 1990 | 5.4         | 40              | 51          | PG        |
| Boiling Point             | 1990 | 6.7         | 94              |             | Not Rated |
| The Adventures of Ford... | 1990 | 6.4         | 25              | 24          | R         |
| Agneepath                 | 1990 | 7.6         |                 |             | Not Rated |

Figure 2: Movies Page

## </movies/data/{imdbID}>

This endpoint is used to get data for a specific movie that is selected via the movies page (the endpoint above). The endpoint uses similar commands to the movies page as it uses a fetch command to get data and stores the data as a 'json'. The main difference between the pages is the way that the information from the 'json' is being used and displayed. The URL takes the form of:

<http://sefdb02.qut.edu.au:3000/movies/data/ID>

This endpoint queries the server by using the 'imdbID' taken from the previous page/endpoint and using it to find even more information on a singular movie.

**Boiling Point**

Year: 1990  
Runtime: 96 Minutes  
Country: Japan  
Box Office: \$1171  
Genres: Action, Comedy, Crime  
Plot: Two disquieted junior baseball players seek revenge on the local yakuza for attacking their coach.

| Role            | Name               | Characters     |
|-----------------|--------------------|----------------|
| actor           | Makoto Ashikawa    | Akira          |
| actor           | Takeshi Kitano     | Uehara         |
| actor           | Yūrei Yanagi       | Masaki         |
| actress         | Yuriko Ishida      | Sayaka         |
| actor           | Taka Guadalcana    | Takashi Iguchi |
| producer        | Toshio Nabeshima   |                |
| cinematographer | Katsumi Yanagijima |                |
| editor          | Toshio Taniguchi   |                |
| actor           | Duncan             | Kazuo          |
| actress         | Eri Fuse           | Fumiyo         |

Internet Movie Database: 6.7  
Rotten Tomatoes: 91  
Metacritic:

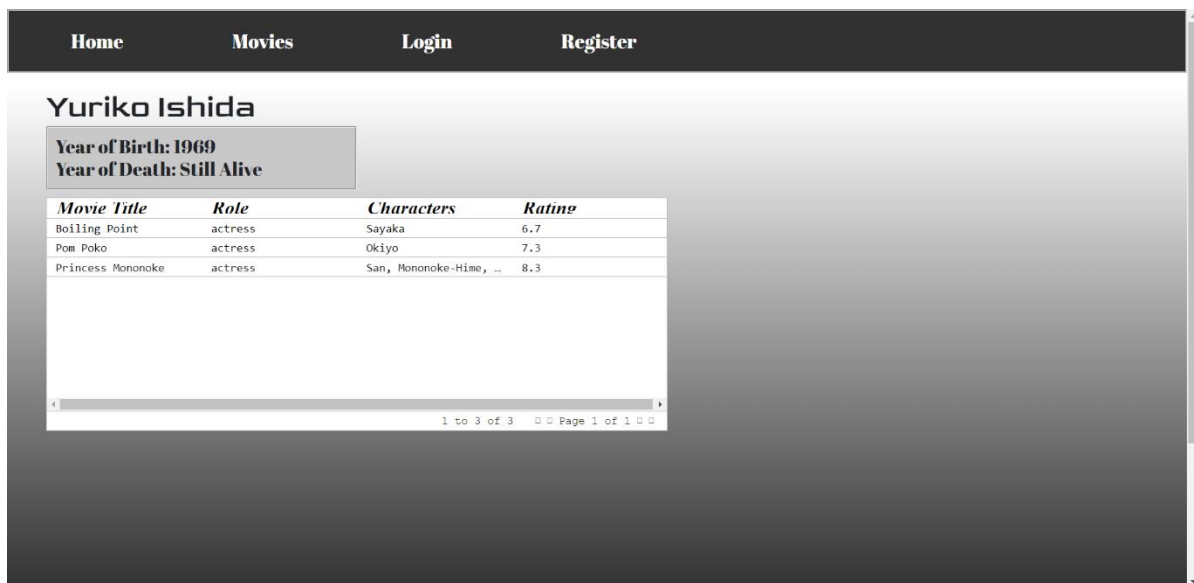
Figure 3: Movie Page

## [/people/{id}](#)

This endpoint is used to get data for a specific person that is selected via the movie page (the endpoint above). The endpoint uses the same commands as the movie page as it uses a fetch command to get data, stores the data as a 'json', and even uses the same method to display the information. Although it uses the same commands, it is locked behind an 'Json' Web Token (JWT), which only allows people with a bearer token to see the data. The URL takes the form of:

<http://sefdb02.gut.edu.au:3000/people/ID>

This endpoint queries the server by using the 'ID' taken from the previous page/endpoint and using it to find even more information on a person, including other films they have appeared in.



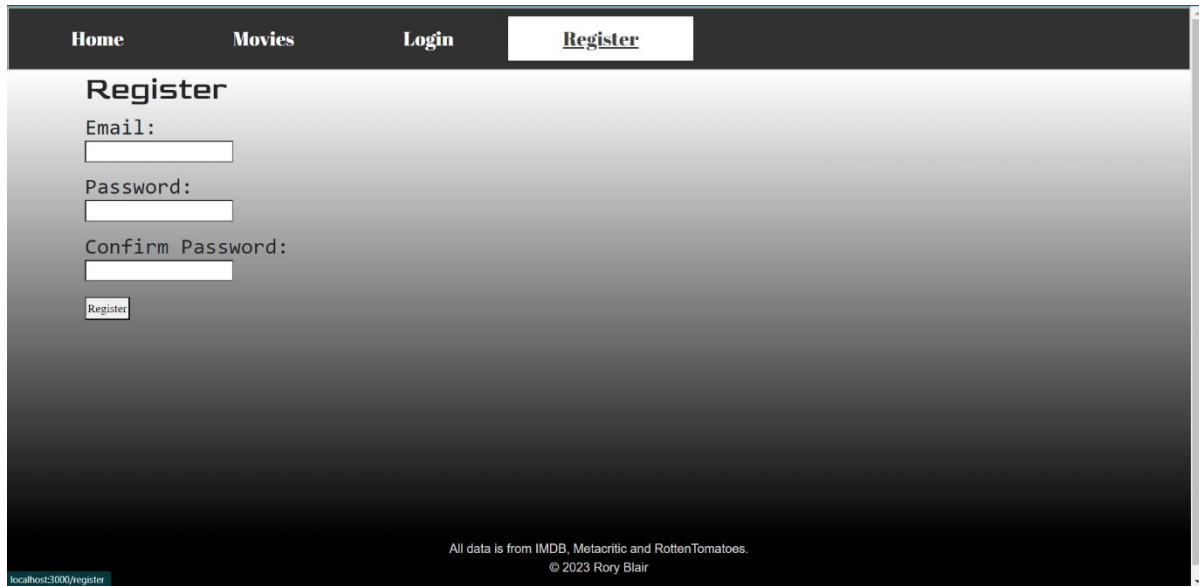
| Movie Title       | Role    | Characters              | Rating |
|-------------------|---------|-------------------------|--------|
| Boiling Point     | actress | Sayaka                  | 6.7    |
| Pom Poko          | actress | Okiyo                   | 7.3    |
| Princess Mononoke | actress | San, Mononoke-Hime, ... | 8.3    |

Figure 4: Person Page

## ***/user/register***

This endpoint is used to register a user into the application. It fetches the data from the server to see if any other user has the same details as the user trying the register. If both passwords are the same, they will be submitted and the user would be redirected to the login page. The URL takes the form of:

<http://sefdb02.qut.edu.au:3000/user/register>



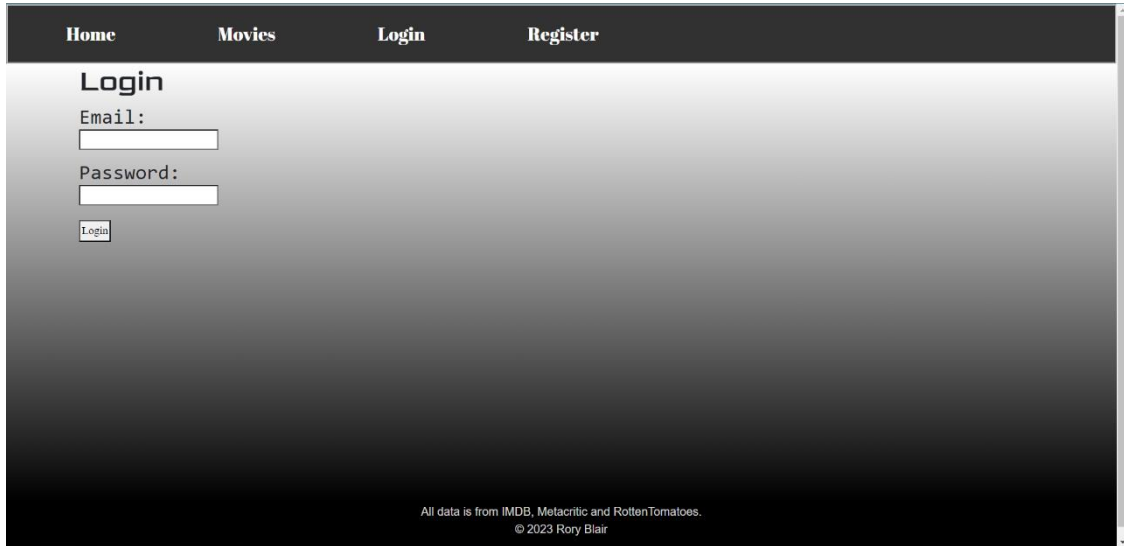
The screenshot shows a web application interface with a dark navigation bar at the top containing links for Home, Movies, Login, and Register. The Register link is highlighted. Below the navigation bar, the page has a light gray gradient background. The title 'Register' is displayed in a bold, black font. Below the title, there are three input fields: 'Email:', 'Password:', and 'Confirm Password:'. Each field is followed by a white text input box. Below the input fields is a small, rectangular 'Register' button. At the bottom of the page, there is a small footer area with the text 'All data is from IMDB, Metacritic and RottenTomatoes.' and '© 2023 Rory Blair'. In the bottom left corner, a small status bar shows 'localhost:3000/register'.

*Figure 5: Register Page*

## ***/user/login***

This endpoint is used to login a user into the application. It fetches the data from the server to see if the user is registered into the system, whilst also getting a JWT for authentication purposes. The URL takes the form of:

<http://sefdb02.qut.edu.au:3000/user/login>

A screenshot of a web application's login page. At the top, there is a dark navigation bar with four links: 'Home', 'Movies', 'Login', and 'Register'. The 'Login' link is highlighted. Below the navigation bar, the page has a light gray background. The word 'Login' is displayed in a bold, black font. Underneath, there are two input fields: 'Email:' and 'Password:'. Each field has a small white rectangular input box. Below the password field is a small, light gray button labeled 'Login'. At the bottom of the page, there is a small line of text: 'All data is from IMDB, Metacritic and RottenTomatoes. © 2023 Rory Blair'.

*Figure 6: Login Page*

## ***/user/refresh***

This endpoint is used to refresh the JWT as the original token created when a user logs in only lasts 5 minutes, meaning a refresh would need to take place so the user isn't logged out. The JWT comes with a refresh token, which is used to refresh the JWT, meaning it wouldn't sign the user out. This endpoint would not have a page, but simply run in the background. The URL takes the form of:

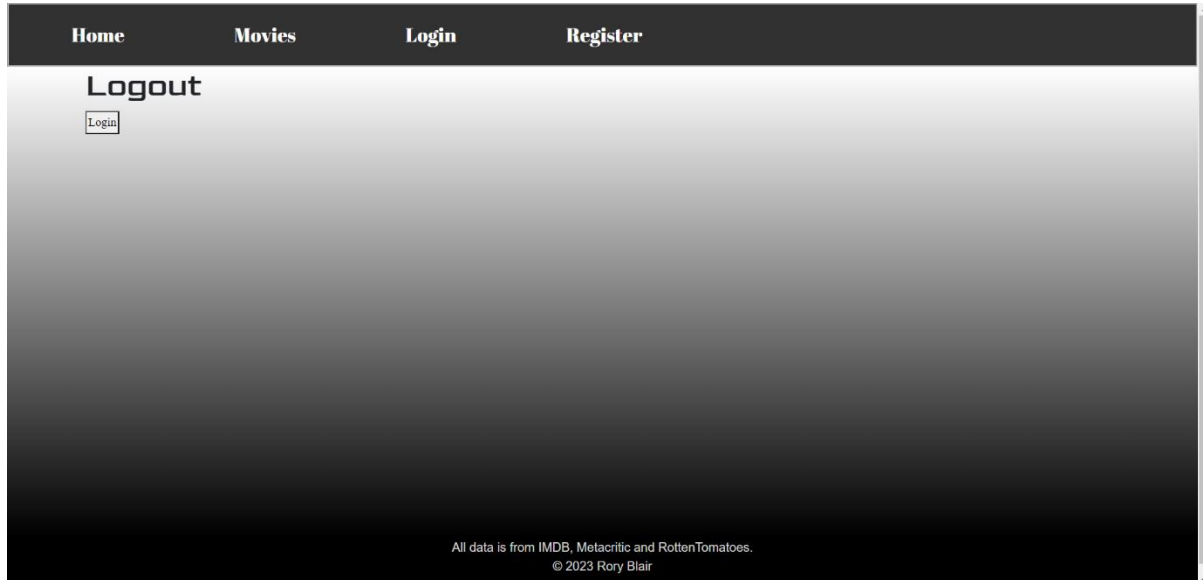
<http://sefdb02.qut.edu.au:3000/user/refresh>



## ***/user/logout***

This endpoint is used to log the user out when they want to, as appose to the JWT running out of time and signing them out. The URL takes the form of:

<http://sefdb02.qut.edu.au:3000/user/logout>



*Figure 7: Logout Page*

# Modules Used

## Module 1: Ag-Grid

### **Description:**

‘Ag-Grid’ is a JavaScript data grid, which is very customisable and fully detailed. It also integrates seamlessly with all major JavaScript architecture and provides magnificent performance (NPM, 2023). Within ‘ag-grid’ there are more modules that are accessible. The two that were used below. ‘Ag-grid-React’ is used as the rendering engine for react, which ‘ag-grid-community’ stores the main logic of the grid (AG Grid, 2023). These two modules are used to make the process of creating tables easier.

### Module 1.1: Ag-Grid-Community

**Docs:** <https://www.npmjs.com/package/ag-grid-community>

### Module 1.2: Ag-Grid-React

**Docs:** <https://www.npmjs.com/package/ag-grid-react?activeTab=readme>

## Module 2: Bootstrap

### **Description:**

Bootstrap is a non-proprietary, effective, feature-packed frontend development library (Zola, 2022). This module will be used to help with the styling and responsiveness of the application webpages.

**Docs:** <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

## Module 3: React

### **Description:**

React is a non-proprietary JavaScript library and framework. It is used for creating web applications and interactable UI quickly and efficiently with less code used compared to normal JavaScript (Herbert, 2022). This module will be used to compile all the different types of files to produce a web application.

**Docs:** <https://legacy.reactjs.org/docs/getting-started.html>

## Module 4: React-Bootstrap

### **Description:**

React-Bootstrap is a more ordered frontend development module as it takes the framework from Bootstrap and uses React components in the place of JavaScript Components. This means that you need no other dependencies to change JavaScript components (Pluralsight, 2022). This application will be used for the buttons.

**Docs:** <https://react-bootstrap.github.io/components/alerts>

## Module 5: React-Dom

### Description:

‘React-Dom’ is a framework that offers ‘Documents Object Model’ (DOM) particular functions, which can be used at a top tier level web application development. It offers methods of changing the DOM, via the use of ‘virtualDOM’ which is used to improve the performance of the web application (Mittal, 2022). This module is used to update the whole web application during the development stage.

**Docs:** <https://www.npmjs.com/package/react-dom>

## Module 6: React-Router-Dom

### Description:

‘React-Router-Dom’ is a fully-featured server-side and client-side routing framework toolkit for React, which allows users to execute dynamic routing in a web application. This allows user to create single-page applications, an application with many components/pages but only uses one page that is dynamic (salehmubashar, 2021). This module will be used to only use one html page that updates according to the user’s input.

**Docs:** <https://www.npmjs.com/package/react-router-dom>

## Module 7: React-Scripts

### Description:

‘React-Scripts’ is a group of functions from the ‘create-react-app’ beginner pack, which let users start developing web applications without configuring. If need be, the ‘react-scripts start’ sets starts a server and sets up an environment for developing web applications (Sakhniuk, 2022). This module will be used to start up the client-side web application on either ‘localhost’ or your own private network (127.0.0.1).

**Docs:** <https://www.npmjs.com/package/react-scripts>

## Module 8: Reactstrap

### Description:

‘Reactstrap’ is a component framework for React that contains a collection of reusable functions used in applications. It is similar to Bootstrap (Module 2) but uses different JavaScript Components and CSS classes (Ijaz, 2022). This module will also be used to style the web pages.

**Docs:** <https://www.npmjs.com/package/reactstrap>

# Application Design and Usability

## Navigation and Layout

### Design Process, choices made and Alternatives:

The design process started as to find which theme I was going to use. The first option was a typical black background with blue details to contrast the background, like black and red with Netflix. The second option was to use a black, white, and grey colour scheme like how the first film was developed in black and white. The second option was chosen as it uses less colours and the black and white contrast each other very well, unlike a mixture of some colours. To give it that antique black and white look, many different fonts were also used, such as 'American Typewriter' and 'Abril Fatface'.

A choice that was made was about how users would be able to search movies by year. An input box was used as you are able search any year. The alternative to this was to use a dropdown menu to search for movies via a particular movie. This alternative was not used as there are many different years and it would be easier to just search for the year, not scroll through a dropdown menu.

Another choice made was to have multiple pages within the movies table, which allows less data to be retrieved at once (it returns 100 results per page). The alternative would be to have one massive page within the movies table, which wasn't used as it would need to retrieve a lot of data (could be over 1000 results on one page). All of these design choices make the web application more antique and simple, like how the first film is considered now. The navigation of a typical use of this web application is shown in figure 8 below.

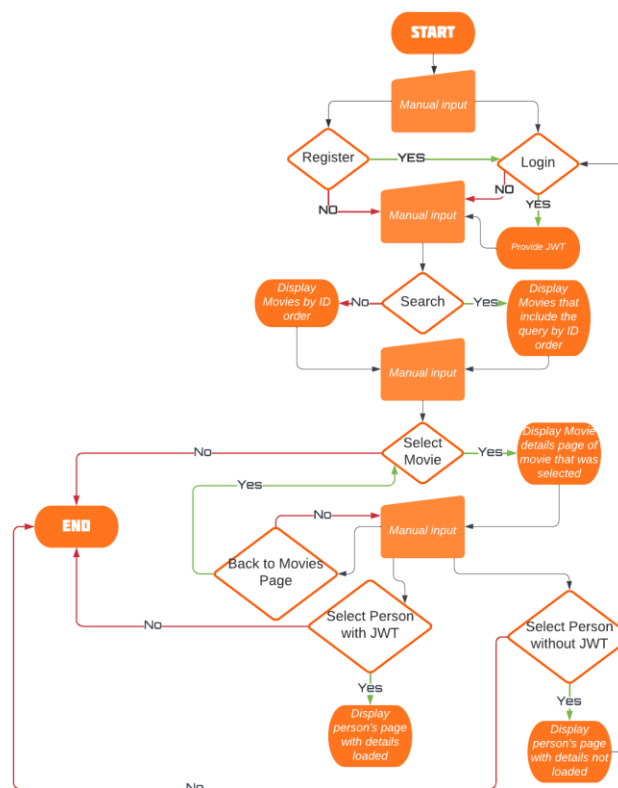
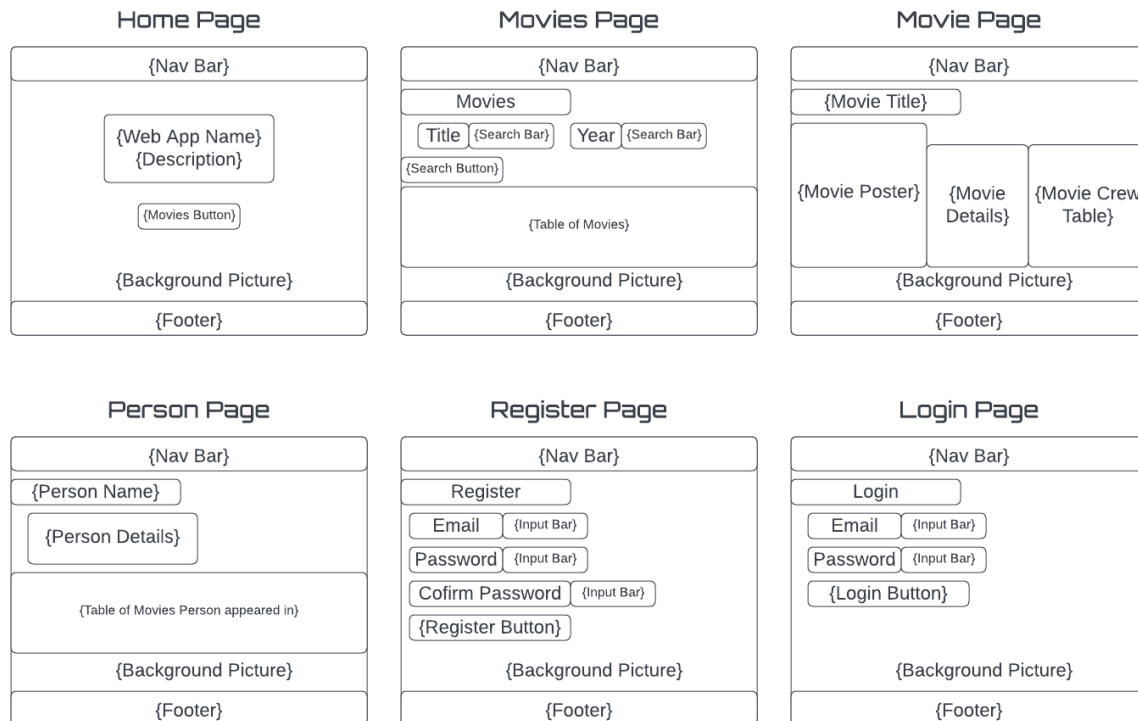


Figure 8: Navigation Flow

## Mock-ups:

Like other design processes mock-ups were created and used to come to a final design concept. This final design has not differed very much compared to the initial mock designs as seen below in figure 9.



*Figure 9: Mock-Ups of Main Pages Used*

## Usability and Quality of Design

### Is your display organized:

The design more antique and simpler is organized with all data required being displayed in a clean, well-organised manner. The data that returns an array is displayed in an ag-grid table, most of the time (except for movie genres). The data that does not return as an array is simply displayed on the screen with there related data type before the data.

### Is the navigation clear:

The navigation through the pages of this web application are understandable and clear. Most of the navigation is done via buttons and clicking on row within tables. Buttons such as the register/login pages which both login or register and redirect to a different page. Clicking on a row of a table will navigate the user to a particular movie, if on the movies page or a person's page, or a particular person, if on a particular movie page. Navigation can also be achieved by using the nav bar at the top of every page.

### Is the visual design consistent across screens(fonts and colours):

The visual design across all pages are consistent with the only noticeable differentiation being the home page which has a picture of a classic movie camera faded within the background. The typical colour scheme used was black, white, and different shades of grey, with the only exceptions being the movie posters that are rendered in. There are 9 different fonts used throughout the application.

(Note: fonts with intext references have been taken from the google fonts API)

- 'Abril Fatface' a google font, was used for the nav bar and the details of movies or person (TypeTogether, 2023).
- 'Merriweather' was used for the home page (Sorkin Type, 2023).
- 'Times New Roman' is used for the column titles in tables.
- 'Goldman' is used for the page titles in the top left corner (Jaikishan Patel, 2023).
- 'Monaco' is used for subtitles on pages.
- 'Josefin Sans' is used for the plot and genres of a particular movie (Santiago Orozco, 2023).
- 'Andale mono' is used for the separate cells within a table, not the column titles.
- 'Courier New' is used for the pagination menu at the bottom of tables.
- 'American Typewriter' is used for the buttons.
- 'Arial' is used for the input boxes.

## Accessibility

### **Provide text equivalent for non-text component:**

This application does provide text equivalents if non-text components fail to load. An example of this is if a movie poster doesn't render, it will be replaced with text saying, 'movie poster'. Another example would be if a table does not have any data to display or authentication has not been met, the tables will say loading, whilst the other details will say unknown.

### **Ensure that information conveyed with colour is also available without colour:**

This application will always convey all information that was present as it uses a simple colour scheme (Black, White, Greys). The only problem that would occur would be with the movie posters, which come in a variety of different colour schemes, bright or dull colours. Even with just one colour, information would still be able to be conveyed as there are also borders or understandable spaces between data, allowing users to differentiate between columns and rows of a table.

### **Organize documents so they may be read without style sheets:**

This application would still be able to be understood without a style sheet as most of the styling done was to change fonts of particular components. As said above, the spacing of data and borders would also help differentiate data if no styling sheet was present.

### **Ensure text equivalents are updated when content changes:**

This application can update text equivalents such as tables, a particular movie's page and a particular person's page. The table on the movies page can update based on the inputs given from the input boxes. There is only one movie page created, that changes depending on the movie ID that is queried. There is only one person page created, that changes depending on the person ID queried.

# Technical Description

## Architecture

The architecture of this web application is split into two main sections, a folder named public and a folder named 'src'. The public folder is used to create a development environment where it creates a client-side rendition of the web application that runs on a private network or 'localhost'. Within 'index.html' is where the google fonts and 'bootstrap' are imported as they must be applied over the entire web application.

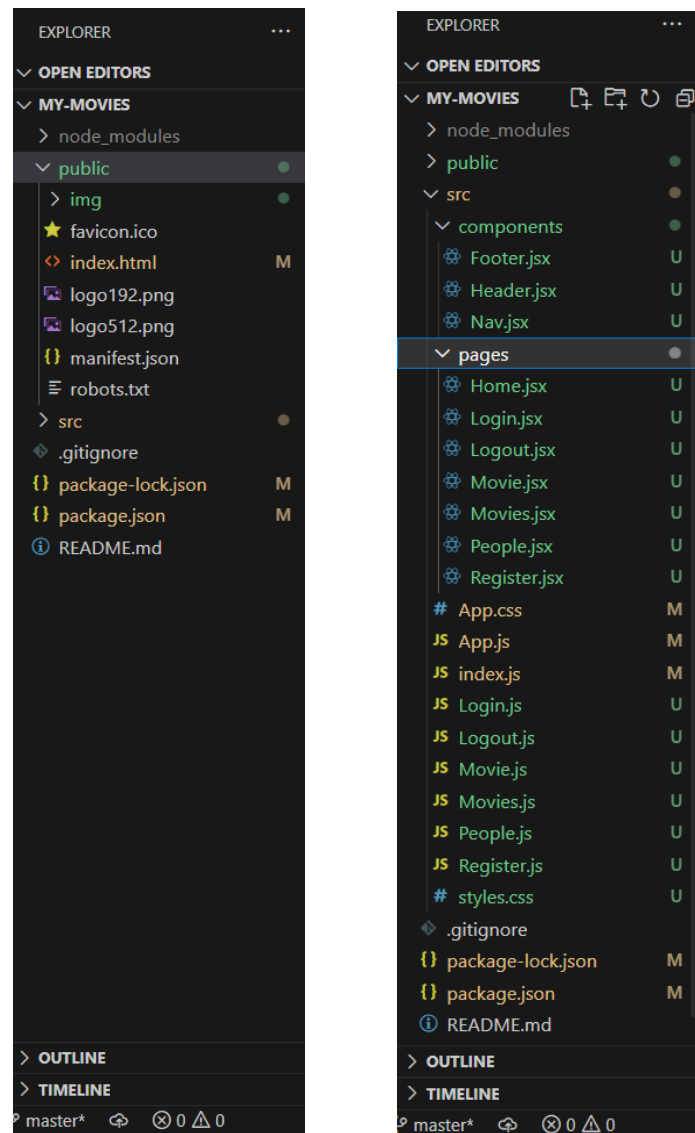


Figure 10: Public Folder Contents (Left) | SRC Folder Contents (Right)



The 'src' folder is where all the components are created and compiled into 'App.js', which is sent to the public folder so it may be rendered. Since most of the functionality is in 'src' it will be discussed in more detail below:

- The Components folder contains the components that are rendered on every page of the web application (Footer and Nav Bar).
  - 'Footer.jsx' is where the content of the footer are created and styled.
  - 'Header.jsx' is where the nav bar is imported into the header of every page.
  - 'Nav.jsx' is where the nav bar is created and styled.
- The Pages folder contains all the separate pages that are used throughout the web application.
  - 'Home.jsx' is where the home page contents are created, such as the faded background and the 'Our Promises' section.
  - 'Login.jsx' imports 'Loggin' from 'Login.js' and renders it on '/login'
  - 'Logout.jsx' imports 'Logout' from 'Logout.js' and renders it on '/logout'
  - 'Movie.jsx' imports 'SingleMovie' from 'Movie.js' and renders it on '/movie?title={Insert ID Here}'
  - 'Movies.jsx' imports 'AppMovie' from 'Movies.js' and renders it on '/movies'
  - 'People.jsx' imports 'Person' from 'People.js' and renders it on '/people?title={Insert ID Here}'
  - 'Register.jsx' imports 'Reggister' from 'Register.js' and renders it on '/register'
- 'App.css' is a styling sheet that is mainly used for the home page.
- 'App.js' creates a function called 'App' that compiles all the 'js' files (except for index.js) and creates navigate routes (Pages) for the 'js' files that require a web page.
- 'index.js' is where the 'App' function from 'App.js' is imported and used to create the entire web application.
- 'Login.js' creates a function called 'Loggin' that creates the login process. It calls a particular URL which gives out a JWT token and returns a html page which is used to render the login page within 'Login.jsx'
- 'Logout.js' creates a function called 'Logout' that creates the logout process. It calls for a JWT and replaces the authorized local storage variable with nothing, therefore sign the user out. this function also returns a html page which is used to render the logout page within 'Logout.jsx'
- 'Movie.js' creates a function called 'SingleMovie' that creates the movie details. It request data for a particular movie and saves that data as a 'json'. That 'json' is then used to retrieve information and return a html page that is used to render the movie page within 'Movie.jsx'
- 'Movies.js' creates a function called 'AppMovie' that creates two search bars and a table. This table is used to show all the data on all movies. The two search bars are used to query the data within the table. Like the other 'js' files, this function also returns a html page that is used to render the movies page within 'Movies.jsx'
- 'Register.js' creates a function called 'Reggister' that allows users to create profiles. The function uses an endpoint to figure out if a user is already register, or not. It also returns a html page, which is used to render the register page within 'Register.jsx'
- 'styles.css' is another styling sheet that is used for all of the web page, even some of the home page.

## Test Plan

| Task                                                                   | Expected Outcome                                                     | Actual Outcome                                                                                                                                                                         | Evidence    |
|------------------------------------------------------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Search for movies by a title but not a year.                           | Movies that include the particular query in the title are returned.  | Movies with that particular query are returned                                                                                                                                         | Appendix 1  |
| Search for movies by a year but not a title.                           | Movies that include the particular query in the title are returned.  | Movies with that particular query are returned                                                                                                                                         | Appendix 2  |
| Search for movies by a title and year.                                 | Movies that include the particular query in the title are returned.  | Movies with that particular query are returned                                                                                                                                         | Appendix 3  |
| Able to view detail on a particular movie on one page.                 | Movie details are displayed                                          | Movie details are displayed with principals (people) being displayed in a table                                                                                                        | Appendix 4  |
| Unable to view detail on a particular person because of authentication | The details will not load on the page                                | Details did not load, instead informing the user to sign in to continue.                                                                                                               | Appendix 5  |
| Able to view details on a particular person because of authentication  | The details will load as the user is authenticated                   | Details did load                                                                                                                                                                       | Appendix 6  |
| Trying to register a user using two different passwords                | Nothing will happen                                                  | Nothing happened                                                                                                                                                                       | Appendix 7  |
| Trying to register a user using the same password                      | It will be redirected onto the login page                            | User is redirected to the login page                                                                                                                                                   | Appendix 8  |
| Logging in general                                                     | It will be redirected to the home page and be given a JWT            | Does not log user in but does supply the user with a JWT                                                                                                                               | Appendix 9  |
| Search for movies by a decade (using 3 numbers)                        | Movies that include the particular query in the decade are returned. | It will send a request and receives a response. The response says the request is a '400 BAD REQUEST' and returns an error message that say that the query must be in a certain format. | Appendix 10 |

## Problems

| Problem Type     | Details / Solution                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Difficulties     | <p><b>Details:</b><br/>A difficulty that kept occurring was that the movie page would only load the movie id but not the other data from the 'json'.</p> <p><b>Solution:</b><br/>Using the console in the inspect page of the browser showed that no call to request the data was made. This was solved by simply calling the function after it was defined</p>                                                                                                                                                                                         |
| Difficulties     | <p><b>Details:</b><br/>A difficulty that kept occurring was that the movie page would only load the movie id but not the other data, even when the 'json' was being printing in the console.</p> <p><b>Solution:</b><br/>Asking for help from the tutor, it was discovered that the function that requested the data, wasn't returning the data to outside of the function, but was keeping it within itself. The solution was to make sure a return variable was created before the call and that the data was stored in that particular variable.</p> |
| Exclusion        | <p><b>Details:</b><br/>A decision that had to be made was the exclusion of a chart for each person's page.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Difficulties     | <p><b>Details:</b><br/>A difficult that was encounter was that a first the person's page was not loading any details on the page and the table just said loading.</p> <p><b>Solution:</b><br/>Using the console in the inspect page of the browser showed that the request was being made but wasn't allowed to respond as authentication was needed. The problem was solved by creating a login page that gave users a token, allowing them authentication, thus the person's page was showing details and the table loaded.</p>                       |
| Persistent error | <p><b>Details:</b><br/>A persistent error that is occurring is to do with the nav bar changing depending on if a user is logged in or not. If a user is not logged in, it should show home, movies, login and register in the nav bar. If a user is logged in, it should show home, movies and logout in the nav bar. Attempts have been made within 'nav.jsx' (where nav bar is created), 'App.js' (where nav bar is loaded into the web app) and 'index.html' (where the app is rendered) but none were successful.</p>                               |

# User Guide

**Step 0).**

# References

AG Grid, 2023. *Get Started with AG Grid*. [Online]

Available at: <https://www.ag-grid.com/react-data-grid/getting-started/#:~:text=ag%2Dgrid%2Dcommunity%20is%20the,and%20their%20versions%20must%20match.>

[Accessed 10 05 2023].

Herbert, D., 2022. *What is React.js? (Uses, Examples, & More)*. [Online]

Available at: <https://blog.hubspot.com/website/react-js>

[Accessed 10 05 2023].

Ijaz, A., 2022. *Reactstrap Vs. React-Bootstrap*. [Online]

Available at: <https://medium.com/geekculture/reactstrap-vs-react-bootstrap-9bd3250809aa#:~:text=Reactstrap%20is%20a%20component%20library,responsive%20websites%20and%20web%20applications.>

[Accessed 10 05 2023].

Jaikishan Patel, 2023. *Goldman*. [Online]

Available at: <https://fonts.google.com/specimen/Goldman/about?query=Goldman>

[Accessed 11 05 2023].

Mittal, P., 2022. *What is DOM in React?*. [Online]

Available at: <https://www.tutorialspoint.com/what-is-dom-in-react#:~:text=DOM%20stands%20for%20'Documents%20Object,updated%20to%20reflect%20the%20change.>

[Accessed 10 05 2023].

NPM, 2023. *ag-grid-community*. [Online]

Available at: <https://www.npmjs.com/package/ag-grid-community>

[Accessed 10 05 2023].

Pluralsight, 2022. *Understanding React-Bootstrap and Making It Work for You*. [Online]

Available at:

[Accessed 10 05 2023].

Sakhniuk, M., 2022. *What is react scripts?*. [Online]

Available at: <https://iq.js.org/questions/react/what-is-react-scripts>

[Accessed 10 05 2023].

salehmubashar, 2021. *What is react-router-dom ?*. [Online]

Available at: <https://www.geeksforgeeks.org/what-is-react-router-dom/>

[Accessed 10 05 2023].

Santiago Orozco, 2023. *Josefin Sans*. [Online]

Available at: <https://fonts.google.com/specimen/Josefin+Sans/about?query=Josefin>

[Accessed 11 05 2023].

Sorkin Type, 2023. *Merriweather*. [Online]

Available at: <https://fonts.google.com/specimen/Merriweather?query=Merri>

[Accessed 11 05 2023].

TypeTogether, 2023. *Abril Fatface*. [Online]

Available at: <https://fonts.google.com/specimen/Abril+Fatface?query=Abril>  
[Accessed 11 05 2023].

Zola, A., 2022. *Bootstrap*. [Online]

Available at:

<https://www.techtarget.com/whatis/definition/bootstrap#:~:text=Bootstrap%20is%20a%20free%2C%20open,of%20syntax%20for%20template%20designs.>

[Accessed 10 05 2023].

# Appendices

## Appendix 1: Search by title

[Home](#) [Movies](#) [Login](#) [Register](#)

### Movies

Title:

Year:

Search

| Title                       | Year | IMDB Rating | Rotten Tomatoes | Metacritics | Rated |
|-----------------------------|------|-------------|-----------------|-------------|-------|
| Star Wars: Episode I - ...  | 1999 | 6.5         | 51              | 51          | PG    |
| Star Wars: Episode II - ... | 2002 | 6.6         | 65              | 54          | PG    |
| Star Wars: Episode III ...  | 2005 | 7.6         | 79              | 68          | PG-13 |
| Star Wars: The Clone Wa...  | 2008 | 5.9         | 18              | 35          | PG    |
| Star Wars: Episode VII ...  | 2015 | 7.8         | 93              | 80          | PG-13 |
| Star Wars: Episode VIII...  | 2017 | 6.9         | 91              | 84          | PG-13 |
| Star Wars: Episode IX - ... | 2019 | 6.5         | 52              | 53          | PG-13 |
| Rogue One: A Star Wars ...  | 2016 | 7.8         | 84              | 65          | PG-13 |
| Solo: A Star Wars Story     | 2018 | 6.9         | 69              | 62          | PG-13 |

## Appendix 2: Search by year

[Home](#) [Movies](#) [Login](#) [Register](#)

### Movies

Title:

Year:

Search

| Title                      | Year | IMDB Rating | Rotten Tomatoes | Metacritics | Rated     |
|----------------------------|------|-------------|-----------------|-------------|-----------|
| Varisu                     | 2023 | 6.1         |                 |             | N/A       |
| Pathaan                    | 2023 | 6.4         |                 | 47          | Not Rated |
| Mission Majnu              | 2023 | 7.2         |                 |             | TV-MA     |
| Thunivu                    | 2023 | 6.2         |                 |             | N/A       |
| Waltair Veerayya           | 2023 | 6.4         |                 |             | N/A       |
| Kuttey                     | 2023 | 6.4         |                 |             | N/A       |
| The Hatchet Wielding Hi... | 2023 | 6.2         |                 |             | TV-MA     |
| Operation Fortune: Ruse... | 2023 | 6.4         | 51              | 50          | R         |

## Appendix 3: Search by title & year

[Home](#) [Movies](#) [Login](#) [Register](#)

### Movies

Title:

Year:

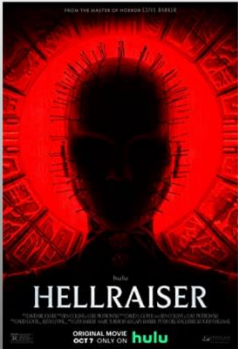
Search

| Title         | Year | IMDB Rating | Rotten Tomatoes | Metacritics | Rated |
|---------------|------|-------------|-----------------|-------------|-------|
| Mission Majnu | 2023 | 7.2         |                 |             | TV-MA |

## Appendix 4: Particular Movie

[Home](#) [Movies](#) [Login](#) [Register](#)

### Hellraiser



**Year:** 2022  
**Runtime:** 121 Minutes  
**Country:** United States, Serbia  
**Box Office:** \$0  
**Genres:**  
Horror  
Mystery  
Thriller  
**Plot:**  
A take on Clive Barker's 1987 horror classic where a young woman struggling with addiction comes into possession of an ancient puzzle box, unaware that its purpose is to summon the Cenobites.

| Role     | Name            | Characters |
|----------|-----------------|------------|
| producer | Keith Levine    |            |
| actress  | Odessa A'zion   | Riley      |
| actress  | Jamie Clayton   | The Priest |
| actor    | Adam Faison     | Colin      |
| actor    | Drew Starkey    | Trevor     |
| director | David Bruckner  |            |
| writer   | Ben Collins     |            |
| writer   | Luke Piotrowski |            |
| writer   | David S. Goyer  |            |
| writer   | Clive Barker    |            |

1 to 10 of 10 Page 1 of 1

Internet Movie Database: 6  
Rotten Tomatoes:  
Metacritic: 55



## Appendix 5: Person Unable

[Home](#) [Movies](#) [Login](#) [Register](#)

Year of Birth: Unknown  
Year of Death: Unknown

| Movie Title | Role | Characters | Rating |
|-------------|------|------------|--------|
| Loading...  |      |            |        |

0 to 0 of 0 Page 0 of 0

## Appendix 6: Person able

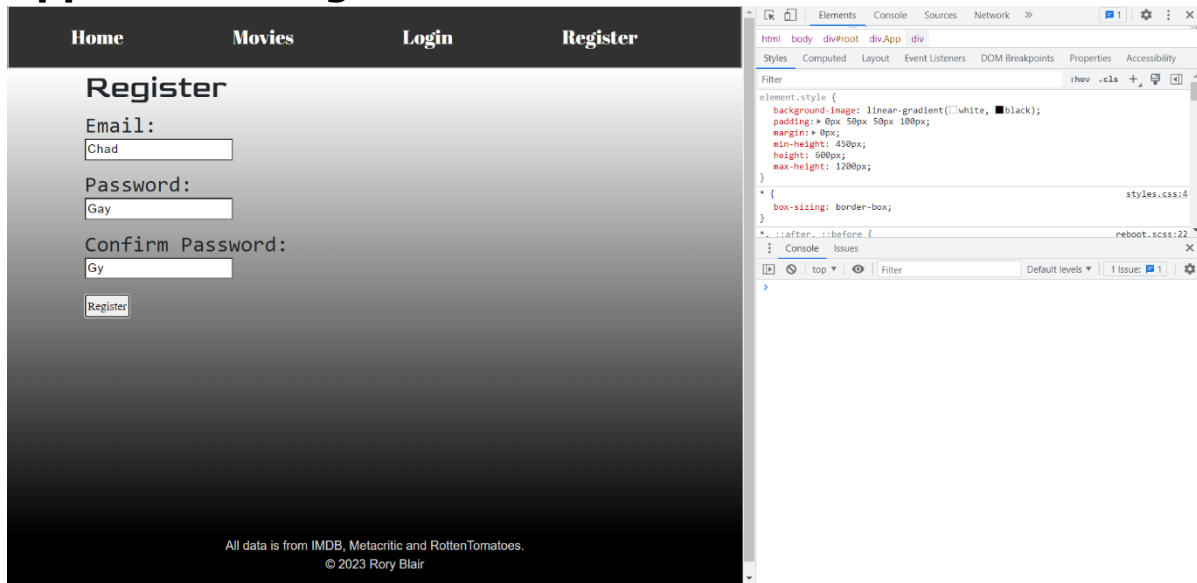
[Home](#) [Movies](#) [Login](#) [Register](#)

Jamie Clayton  
Year of Birth: 1978  
Year of Death: Still Alive

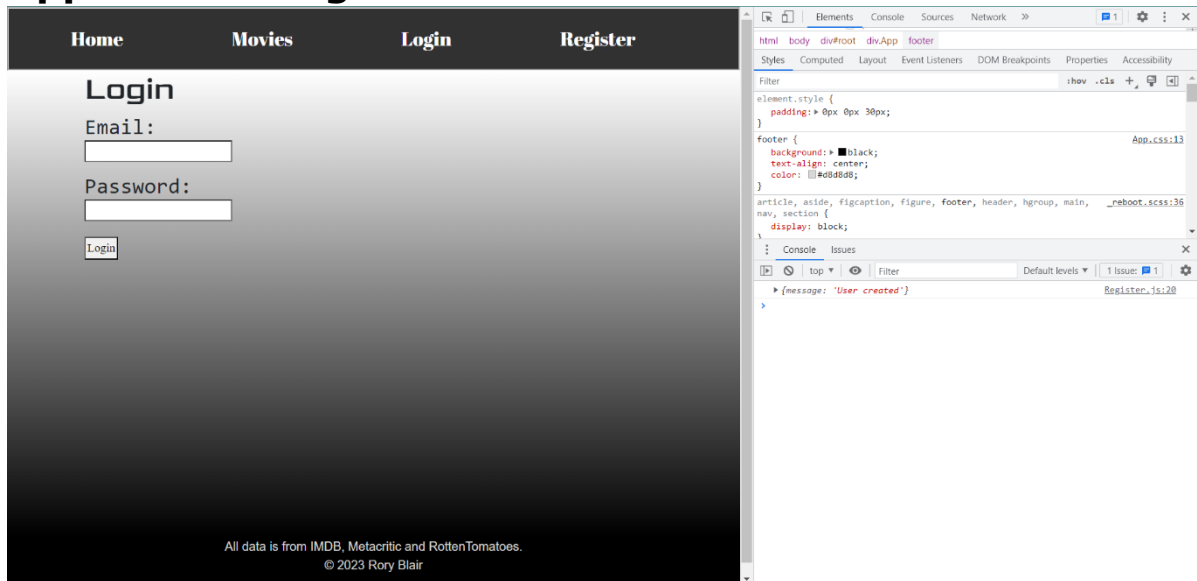
| Movie Title | Role    | Characters | Rating |
|-------------|---------|------------|--------|
| Hellraiser  | actress | The Priest | 6      |

1 to 1 of 1 Page 1 of 1

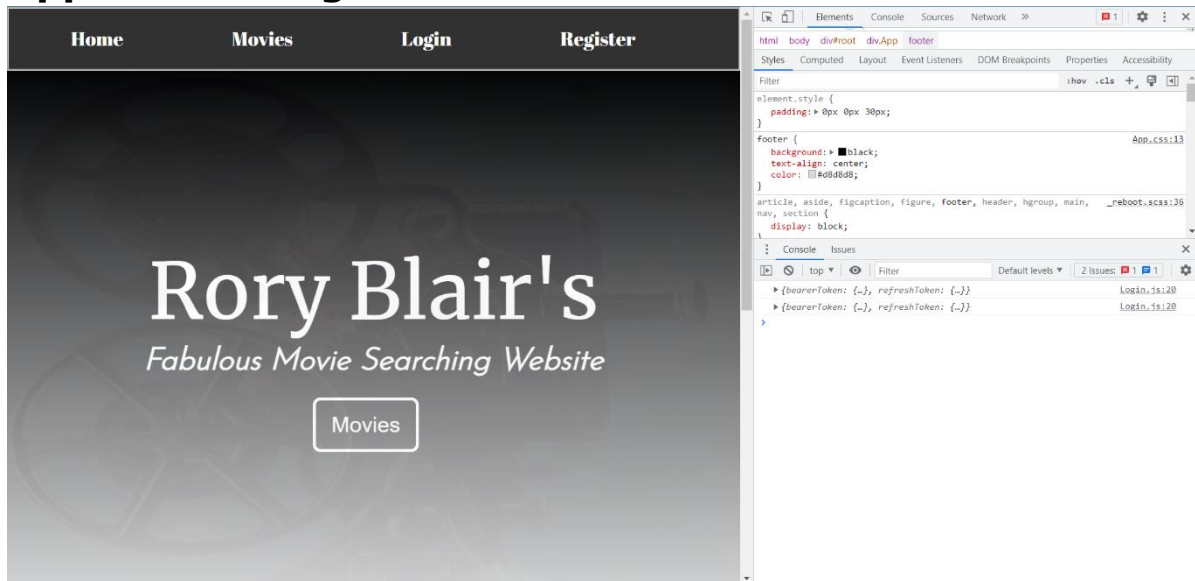
## Appendix 7: Register Unable



## Appendix 8: Register able



## Appendix 9: Login



## Appendix 10: Decade

